COLLEGE ENGINEERING PROJECT · PROJECT 25 OF 55

Multilingual Speech Emotion Recognition Using Deep Learning

Domain: Audio Signal Processing & Deep Learning | **Module:** project_25_speech_emotion

PROJECT ABSTRACT

Human speech communicates not only words but also emotions through variations in tone, pitch, intensity, rhythm, and speaking style. Conventional speech recognition systems mainly focus on converting speech into text and often overlook the emotional state of the speaker. The proposed Multilingual Speech Emotion Recognition System Using Deep Learning aims to identify emotions from spoken audio across multiple languages, enabling more intelligent and human-aware voice interaction. The system accepts speech as input and extracts important acoustic features such as pitch, frequency, energy, and spectral characteristics. Deep learning techniques are then used to learn the relationship between these speech patterns and emotional states such as happiness, sadness, anger, fear, and neutrality. The model is designed to work across multiple languages, making it suitable for multilingual environments where pronunciation, vocabulary, and speaking styles may differ. The major uniqueness of the proposed system is its focus on how a person speaks rather than only what the person says. The system can identify emotional characteristics even when the language changes. It can also be extended to estimate emotion intensity, allowing the system to distinguish between low, medium, and high levels of an emotion instead of providing only a basic emotion label. This makes the solution more meaningful for real-world applications. Another important feature is the possibility of cross-language emotion learning, where emotional patterns learned from one language can assist recognition in another language. Audio preprocessing techniques can also be incorporated to improve recognition when speech contains background noise. With further development, the system can support real-time emotion detection through microphone input. The proposed system can be applied in intelligent virtual assistants, customer-support systems, online education, entertainment, and human-computer interaction. By combining multilingual speech processing, language-independent emotional feature learning, and emotion-intensity recognition, the project aims to create a more context-aware voice intelligence system. Overall, the project demonstrates how deep learning can bridge the gap between speech recognition and emotional understanding, enabling AI systems to recognize not only what a person says, but also the emotional context in which it is spoken.

📌 Project Metadata & Specifications

Project Title	Multilingual Speech Emotion Recognition Using Deep Learning

Domain / Specialization	Audio Signal Processing & Deep Learning
Source Directory	`project_25_speech_emotion`
Backend REST API	`http://localhost:5025` (Flask Python)
Frontend Interface	Interactive Dashboard (`frontend/index.html`)
Execution Mode	Hybrid Dual-Mode (Online API & Offline Autonomous Simulation)

🏗️ System Methodology & Implementation Phases

PROJECT 25 — PHASE 1

Phase 1 — Requirements, Data and Prototype

The first phase will convert the project concept into a measurable prototype for NLP/language or intelligent information processing. The work begins by defining the exact input, output, user flow and evaluation criteria from the project abstract. A small requirements specification will identify the primary prediction/detection task, expected classes or target value, acceptable response time, and the form in which results must be displayed.

1. Problem Definition and Scope

Define the operational scenario, target users, system boundaries, assumptions, success criteria and non-functional requirements. Separate what the model must predict from what the application must display or recommend. Identify cases that should be rejected, flagged for review, or returned with low confidence.

2. Data Preparation

Identify a suitable dataset or approved collection procedure matching the abstract. Build a data dictionary, remove unusable records, handle missing values, normalize or standardize numerical fields where appropriate, and verify labels. For image/video/audio/text projects, define preprocessing, augmentation and quality-control rules. Split data into training, validation and test sets without leakage.

3. Baseline Model and Experiment Setup

Implement a reproducible baseline using the algorithm/model family stated in the abstract when one is specified. Establish a simple reference result before introducing additional complexity. Record dataset version, preprocessing settings, model configuration, random seed, training time and evaluation metrics so that later improvements can be compared objectively.

4. Initial Application Prototype

Build the minimum user-facing workflow needed to demonstrate the core task. The prototype should accept valid input, run preprocessing, invoke the baseline model, and return the predicted result with an understandable confidence/score where supported. Keep model logic separate from the interface so the model can be replaced without rewriting the application.

Phase 1 Deliverables

Requirements and scope note; prepared dataset or data pipeline; baseline model; experiment log; initial application/prototype; and a short Phase 1 validation report documenting what works, what remains uncertain, and the measurements to be improved in Phase 2.

Project-Specific Phase 1 Work Items

• Define text cleaning, tokenization/embedding and document-length handling rules.

• Create representative examples for each output class or information field.
• Track confidence and ambiguous cases for human review rather than forcing uncertain outputs.

PROJECT 25 — PHASE 2

Phase 2 — Optimization, Integration and Validation
The second phase will turn the Phase 1 prototype into a validated project for NLP/language or intelligent information processing. The emphasis is on improving model reliability, connecting the complete workflow, testing realistic edge cases, and producing evidence that the final system meets the defined objectives.
1. Model Improvement
Compare relevant model configurations, tune important hyperparameters, address class imbalance where present, and evaluate whether additional features or augmentation improve generalization. Use validation data for tuning and reserve the test set for final evaluation. Prefer measurable improvements over unnecessary model complexity.
2. End-to-End Integration
Connect the data/input layer, preprocessing, inference engine, result-processing logic, database or file storage where required, and user interface. Add input validation, error handling, logging and clear status messages. If recommendations, alerts, summaries, risk scores or dashboards are part of the abstract, implement them as a separate decision-support layer around the model.
3. Testing and Evaluation
Perform functional, model, integration and usability tests. Evaluate accuracy or regression error as appropriate, along with precision, recall, F1, confusion matrix, ROC-AUC, latency or other task-specific measures where relevant. Test difficult, noisy, incomplete and out-of-distribution inputs and document failure cases instead of hiding them.
4. Demonstration and Final Evidence
Prepare representative test cases, screenshots or result tables, model-comparison evidence and a final demonstration workflow. Document limitations, data constraints, privacy/security considerations where applicable, and future extensions. The final report should distinguish measured results from proposed future work.
Phase 2 Deliverables
Optimized model; integrated application; test and evaluation report; representative evidence; limitations and risk register; deployment/demo instructions; and final project documentation suitable for academic review or demonstration.
Project-Specific Phase 2 Validation
• Compare the final model with the Phase 1 baseline using the same held-out evaluation set.
• Document representative success cases and failure cases instead of reporting only aggregate accuracy.
• Prepare a final demonstration script covering input, processing, output, validation and limitations.
Final Expected Outcome
A reproducible, tested prototype that implements the project concept described in the supplied abstract, together with measurable evaluation evidence and clear documentation of limitations, assumptions and future enhancement opportunities.

🚀 Local Deployment & Execution Guide

Follow the steps below to initialize and demonstrate the system:

1. Backend Server Setup

```
# Navigate to the backend directory cd "/Users/balamurugan/Documents/orbyte globel
/college project/project_25_speech_emotion/backend" # Install project dependencies pip
install -r requirements.txt # Run the Flask API service python3 app.py
```

The backend microservice will listen for requests at `http://localhost:5025`.

2. Frontend User Interface

- Open `/Users/balamurugan/Documents/orbyte globel /college project/project_25_speech_emotion/frontend/index.html` directly in Google Chrome, Firefox, or Safari.
- Alternatively, launch via the central **Master Launchpad Portal (`index.html`)** located at the root directory.
- The user interface is designed with **hybrid dual-mode resilience**: when the backend API is live, real-time inferences are queried; when offline, authentic client-side mathematical models simulate sensor telematics.